

03_sol: Reliability-Focused Incident Triage Agent

Contains:

- the same scenario as `03_mock`
- one complete reference implementation
- grading tests

```
In [ ]: # Chunk overview: Prepare imports, fixtures, and helper scaffolding used by
# Why this chunk exists: it makes the solution easier to reason about under
# Import required modules for this solution step.
import inspect
# Import required modules for this solution step.
import json
# Import required modules for this solution step.
from copy import deepcopy
# Import required modules for this solution step.
from typing import Any, Callable

# Assign computed data to a named variable for later use.
LOOKUP_ATTEMPTS: dict[str, int] = {}

# Define `reset_state` so this step is reusable and testable.
def reset_state() -> None:
    # Call this function to perform the next operation.
    LOOKUP_ATTEMPTS.clear()

# Define `fetch_ticket` so this step is reusable and testable.
def fetch_ticket(ticket_id: str) -> dict[str, Any]:
    # Return the computed value for the caller.
    return {
        # Execute this line as part of the solution flow.
        "ticket_id": ticket_id,
        # Execute this line as part of the solution flow.
        "service": "payments" if ticket_id == "inc-1" else "billing",
        # Execute this line as part of the solution flow.
        "severity": "high",
    }
    # Execute this line as part of the solution flow.

# Define `lookup_runbook` so this step is reusable and testable.
def lookup_runbook(service: str) -> dict[str, Any]:
    # Assign computed data to a named variable for later use.
    count = LOOKUP_ATTEMPTS.get(service, 0)
    # Assign computed data to a named variable for later use.
    LOOKUP_ATTEMPTS[service] = count + 1
```

```

# Check this condition to choose the correct branch.
if service == "payments" and count == 0:
    # Raise explicit error to fail fast on invalid state.
    raise RuntimeError("transient backend timeout")
# Return the computed value for the caller.
return {"service": service, "playbook": f"restart_{service}_workers"}

# Assign computed data to a named variable for later use.
TOOL_REGISTRY: dict[str, Callable[..., Any]] = {
    # Execute this line as part of the solution flow.
    "fetch_ticket": fetch_ticket,
    # Execute this line as part of the solution flow.
    "lookup_runbook": lookup_runbook,
}
# Execute this line as part of the solution flow.
}

# Define class `TriageModel` to organize related behavior.
class TriageModel:
    # Define `__init__` so this step is reusable and testable.
    def __init__(self, scenario: str) -> None:
        # Assign computed data to a named variable for later use.
        self.scenario = scenario
        # Assign computed data to a named variable for later use.
        self.step = 0

    # Define `__call__` so this step is reusable and testable.
    def __call__(self, messages: list[dict[str, Any]]) -> dict[str, Any]:
        # Assign computed data to a named variable for later use.
        self.step += 1

        # Check this condition to choose the correct branch.
        if self.scenario == "cache":
            # Check this condition to choose the correct branch.
            if self.step == 1:
                # Return the computed value for the caller.
                return {
                    # Execute this line as part of the solution flow.
                    "stop_reason": "tool_use",
                    # Execute this line as part of the solution flow.
                    "content": [
                        # Execute this line as part of the solution flow.
                        {"type": "tool_use", "id": "a1", "name": "lookup_rur"},
                        # Execute this line as part of the solution flow.
                    ],
                    # Execute this line as part of the solution flow.
                }
            # Check this condition to choose the correct branch.
            if self.step == 2:
                # Return the computed value for the caller.
                return {
                    # Execute this line as part of the solution flow.
                    "stop_reason": "tool_use",
                    # Execute this line as part of the solution flow.
                    "content": [

```

```

        # Execute this line as part of the solution flow.
        {"type": "tool_use", "id": "a2", "name": "lookup_rur
    # Execute this line as part of the solution flow.
    ],
    # Execute this line as part of the solution flow.
    }
# Return the computed value for the caller.
return {
    # Execute this line as part of the solution flow.
    "stop_reason": "end_turn",
    # Execute this line as part of the solution flow.
    "content": [
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "type": "text",
            # Execute this line as part of the solution flow.
            "text": json.dumps(
                # Execute this line as part of the solution flow
                {"summary": "billing issue mitigated", "action":
            # Execute this line as part of the solution flow.
            ),
            # Execute this line as part of the solution flow.
        }
        # Execute this line as part of the solution flow.
    ],
    # Execute this line as part of the solution flow.
}

# Check this condition to choose the correct branch.
if self.scenario == "retry":
    # Check this condition to choose the correct branch.
    if self.step == 1:
        # Return the computed value for the caller.
        return {
            # Execute this line as part of the solution flow.
            "stop_reason": "tool_use",
            # Execute this line as part of the solution flow.
            "content": [
                # Execute this line as part of the solution flow.
                {"type": "tool_use", "id": "b1", "name": "lookup_rur
            # Execute this line as part of the solution flow.
            ],
            # Execute this line as part of the solution flow.
        }
# Return the computed value for the caller.
return {
    # Execute this line as part of the solution flow.
    "stop_reason": "end_turn",
    # Execute this line as part of the solution flow.
    "content": [
        # Execute this line as part of the solution flow.
        {
            # Execute this line as part of the solution flow.
            "type": "text",
            # Execute this line as part of the solution flow.

```

```

        "text": json.dumps(
            # Execute this line as part of the solution flow.
            {"summary": "payments issue mitigated", "action": "mitigate"},
            # Execute this line as part of the solution flow.
        ),
        # Execute this line as part of the solution flow.
    }
    # Execute this line as part of the solution flow.
],
# Execute this line as part of the solution flow.
}

# Check this condition to choose the correct branch.
if self.scenario == "loop":
    # Return the computed value for the caller.
    return {
        # Execute this line as part of the solution flow.
        "stop_reason": "tool_use",
        # Execute this line as part of the solution flow.
        "content": [{"type": "tool_use", "id": "loop", "name": "fetch_data"}],
        # Execute this line as part of the solution flow.
    }

# Return the computed value for the caller.
return {"stop_reason": "end_turn", "content": [{"type": "text", "text": "The payments issue has been mitigated."}]}

```

```

In [ ]: # Chunk overview: Implement the final reference solution in a clean, stepwise manner.
# Why this chunk exists: it makes the solution easier to reason about under uncertainty.
# Define `_validate_tool_call` so this step is reusable and testable.
def _validate_tool_call(tool_call: dict[str, Any], tool_registry: dict[str, Tool]):
    # Iterate through items to process each element deterministically.
    for key in ("id", "name", "input"):
        # Check this condition to choose the correct branch.
        if key not in tool_call:
            # Return the computed value for the caller.
            return "tool_call_missing_required_fields"

    # Assign computed data to a named variable for later use.
    name = tool_call["name"]
    # Assign computed data to a named variable for later use.
    payload = tool_call["input"]
    # Check this condition to choose the correct branch.
    if name not in tool_registry:
        # Return the computed value for the caller.
        return "unknown_tool"
    # Check this condition to choose the correct branch.
    if not isinstance(payload, dict):
        # Return the computed value for the caller.
        return "tool_input_must_be_object"

    # Assign computed data to a named variable for later use.
    sig = inspect.signature(tool_registry[name])
    # Assign computed data to a named variable for later use.
    missing = [
        # Execute this line as part of the solution flow.
        param.name
        for param in sig.parameters.values()
        if param.kind == param.POSITIONAL_ONLY and param.name not in payload
    ]
    if missing:
        # Return the computed value for the caller.
        return "tool_input_missing_required_fields"
    # Execute this line as part of the solution flow.
    tool_registry[name].run(payload)
    # Return the computed value for the caller.
    return {"content": [{"type": "text", "text": "Tool call completed successfully."}]}

```

```

        # Iterate through items to process each element deterministically.
        for param in sig.parameters.values()
        # Check this condition to choose the correct branch.
        if param.default is inspect._empty and param.name not in payload
        # Execute this line as part of the solution flow.
    ]
    # Check this condition to choose the correct branch.
    if missing:
        # Return the computed value for the caller.
        return f"missing_required_args:{','.join(sorted(missing))}"
    # Return the computed value for the caller.
    return None

# Define `execute_tool_call` so this step is reusable and testable.
def execute_tool_call(
    # Execute this line as part of the solution flow.
    tool_call: dict[str, Any],
    # Execute this line as part of the solution flow.
    tool_registry: dict[str, Callable[..., Any]],
    # Execute this line as part of the solution flow.
    cache: dict[str, dict[str, Any]],
    # Execute this line as part of the solution flow.
) -> dict[str, Any]:
    # Assign computed data to a named variable for later use.
    tool_id = str(tool_call.get("id", "missing_id"))
    # Assign computed data to a named variable for later use.
    tool_name = str(tool_call.get("name", "missing_name"))
    # Assign computed data to a named variable for later use.
    payload = tool_call.get("input", {})
    # Assign computed data to a named variable for later use.
    cache_key = f"{tool_name}:{json.dumps(payload, sort_keys=True)}"

    # Assign computed data to a named variable for later use.
    err = _validate_tool_call(tool_call, tool_registry)
    # Check this condition to choose the correct branch.
    if err:
        # Return the computed value for the caller.
        return {
            # Execute this line as part of the solution flow.
            "type": "tool_result",
            # Execute this line as part of the solution flow.
            "tool_use_id": tool_id,
            # Execute this line as part of the solution flow.
            "is_error": True,
            # Execute this line as part of the solution flow.
            "from_cache": False,
            # Assign computed data to a named variable for later use.
            "content": json.dumps({"error": err}, sort_keys=True),
            # Execute this line as part of the solution flow.
        }

    # Check this condition to choose the correct branch.
    if cache_key in cache:
        # Assign computed data to a named variable for later use.
        cached = deepcopy(cache[cache_key])

```

```
# Assign computed data to a named variable for later use.
cached["tool_use_id"] = tool_id
# Assign computed data to a named variable for later use.
cached["from_cache"] = True
# Return the computed value for the caller.
return cached
```

```
# Assign computed data to a named variable for later use.
```

```
attempt = 0
```

```
# Loop while this condition remains true.
```

```
while True:
```

```
    # Assign computed data to a named variable for later use.
```

```
    attempt += 1
```

```
    # Start guarded block to handle potential runtime errors.
```

```
    try:
```

```
        # Assign computed data to a named variable for later use.
```

```
        result = tool_registry[tool_name](**payload)
```

```
        # Assign computed data to a named variable for later use.
```

```
        msg = {
```

```
            # Execute this line as part of the solution flow.
```

```
            "type": "tool_result",
```

```
            # Execute this line as part of the solution flow.
```

```
            "tool_use_id": tool_id,
```

```
            # Execute this line as part of the solution flow.
```

```
            "is_error": False,
```

```
            # Execute this line as part of the solution flow.
```

```
            "from_cache": False,
```

```
            # Assign computed data to a named variable for later use.
```

```
            "content": json.dumps({"result": result}, sort_keys=True),
```

```
        # Execute this line as part of the solution flow.
```

```
    }
```

```
    # Assign computed data to a named variable for later use.
```

```
    cache[cache_key] = deepcopy(msg)
```

```
    # Return the computed value for the caller.
```

```
    return msg
```

```
# Handle expected failure path and keep behavior predictable.
```

```
except RuntimeError as exc:
```

```
    # Assign computed data to a named variable for later use.
```

```
    transient = "transient" in str(exc).lower()
```

```
    # Check this condition to choose the correct branch.
```

```
    if transient and attempt == 1:
```

```
        # Execute this line as part of the solution flow.
```

```
        continue
```

```
    # Return the computed value for the caller.
```

```
    return {
```

```
        # Execute this line as part of the solution flow.
```

```
        "type": "tool_result",
```

```
        # Execute this line as part of the solution flow.
```

```
        "tool_use_id": tool_id,
```

```
        # Execute this line as part of the solution flow.
```

```
        "is_error": True,
```

```
        # Execute this line as part of the solution flow.
```

```
        "from_cache": False,
```

```
        # Assign computed data to a named variable for later use.
```

```
        "content": json.dumps({"error": str(exc)}, sort_keys=True),
```

```
    # Execute this line as part of the solution flow.
```

```

    }
    # Handle expected failure path and keep behavior predictable.
    except Exception as exc: # pragma: no cover
        # Return the computed value for the caller.
        return {
            # Execute this line as part of the solution flow.
            "type": "tool_result",
            # Execute this line as part of the solution flow.
            "tool_use_id": tool_id,
            # Execute this line as part of the solution flow.
            "is_error": True,
            # Execute this line as part of the solution flow.
            "from_cache": False,
            # Assign computed data to a named variable for later use.
            "content": json.dumps({"error": str(exc)}, sort_keys=True),
            # Execute this line as part of the solution flow.
        }

# Define `parse_final_output` so this step is reusable and testable.
def parse_final_output(output_text: str) -> dict[str, Any]:
    # Assign computed data to a named variable for later use.
    data = json.loads(output_text)
    # Assign computed data to a named variable for later use.
    required = {"summary", "action", "confidence"}
    # Assign computed data to a named variable for later use.
    missing = required.difference(data)
    # Check this condition to choose the correct branch.
    if missing:
        # Raise explicit error to fail fast on invalid state.
        raise ValueError(f"missing_final_keys:{','.join(sorted(missing))}")
    # Check this condition to choose the correct branch.
    if not isinstance(data["confidence"], (int, float)):
        # Raise explicit error to fail fast on invalid state.
        raise ValueError("confidence_must_be_numeric")
    # Return the computed value for the caller.
    return data

# Define `run_agent` so this step is reusable and testable.
def run_agent(
    # Execute this line as part of the solution flow.
    user_prompt: str,
    # Execute this line as part of the solution flow.
    model: Callable[[list[dict[str, Any]]], dict[str, Any]],
    # Execute this line as part of the solution flow.
    tool_registry: dict[str, Callable[..., Any]],
    # Assign computed data to a named variable for later use.
    max_steps: int = 6,
    # Execute this line as part of the solution flow.
) -> dict[str, Any]:
    # Assign computed data to a named variable for later use.
    messages: list[dict[str, Any]] = [{"role": "user", "content": [{"type":
    # Assign computed data to a named variable for later use.
    cache: dict[str, dict[str, Any]] = {}
    # Assign computed data to a named variable for later use.

```

```

stats = {"tool_calls": 0, "cache_hits": 0}

# Iterate through items to process each element deterministically.
for _ in range(max_steps):
    # Assign computed data to a named variable for later use.
    response = model(messages)
    # Assign computed data to a named variable for later use.
    stop_reason = response.get("stop_reason")
    # Assign computed data to a named variable for later use.
    content = response.get("content", [])
    # Check this condition to choose the correct branch.
    if not isinstance(content, list):
        # Raise explicit error to fail fast on invalid state.
        raise RuntimeError("assistant_content_must_be_list")
    # Call this function to perform the next operation.
    messages.append({"role": "assistant", "content": content})

    # Check this condition to choose the correct branch.
    if stop_reason == "tool_use":
        # Execute this line as part of the solution flow.
        tool_calls = [block for block in content if block.get("type") == "tool_use"]
        # Check this condition to choose the correct branch.
        if not tool_calls:
            # Raise explicit error to fail fast on invalid state.
            raise RuntimeError("tool_use_without_blocks")
        # Assign computed data to a named variable for later use.
        tool_results: list[dict[str, Any]] = []
        # Iterate through items to process each element deterministically.
        for tool_call in tool_calls:
            # Assign computed data to a named variable for later use.
            tool_msg = execute_tool_call(tool_call, tool_registry, cache)
            # Assign computed data to a named variable for later use.
            stats["tool_calls"] += 1
            # Check this condition to choose the correct branch.
            if tool_msg.get("from_cache"):
                # Assign computed data to a named variable for later use.
                stats["cache_hits"] += 1
            # Call this function to perform the next operation.
            tool_results.append(tool_msg)
        # Call this function to perform the next operation.
        messages.append({"role": "user", "content": tool_results})
        # Execute this line as part of the solution flow.
        continue

    # Check this condition to choose the correct branch.
    if stop_reason == "end_turn":
        # Assign computed data to a named variable for later use.
        output_text = " ".join(
            # Call this function to perform the next operation.
            block.get("text", "").strip()
            # Iterate through items to process each element deterministically.
            for block in content
            # Check this condition to choose the correct branch.
            if block.get("type") == "text"
        )
        # Call this function to perform the next operation.
        output_text = output_text.strip()

```



```

        # Assign computed data to a named variable for later use.
        final = parse_final_output(output_text or "{}")
        # Return the computed value for the caller.
        return {"final": final, "messages": messages, "stats": stats}

    # Raise explicit error to fail fast on invalid state.
    raise RuntimeError(f"unsupported_stop_reason:{stop_reason}")

# Raise explicit error to fail fast on invalid state.
raise RuntimeError("max_steps_exceeded")

```

```

In [ ]: # Chunk overview: Run checks that prove the implementation meets the problem
# Why this chunk exists: it makes the solution easier to reason about under
# Define `_tool_result_blocks` so this step is reusable and testable.
def _tool_result_blocks(messages: list[dict[str, Any]]) -> list[dict[str, Ar
    # Assign computed data to a named variable for later use.
    blocks: list[dict[str, Any]] = []
    # Iterate through items to process each element deterministically.
    for message in messages:
        # Check this condition to choose the correct branch.
        if message.get("role") != "user":
            # Execute this line as part of the solution flow.
            continue
        # Iterate through items to process each element deterministically.
        for block in message.get("content", []):
            # Check this condition to choose the correct branch.
            if block.get("type") == "tool_result":
                # Call this function to perform the next operation.
                blocks.append(block)
    # Return the computed value for the caller.
    return blocks

# Define `run_exam03_tests` so this step is reusable and testable.
def run_exam03_tests() -> None:
    # Call this function to perform the next operation.
    reset_state()

    # Existing inline note.
    # 1) Cache behavior
    # Assign computed data to a named variable for later use.
    model = TriageModel("cache")
    # Assign computed data to a named variable for later use.
    result = run_agent("triage billing", model, TOOL_REGISTRY)
    # Assert expected behavior to validate correctness.
    assert result["stats"]["cache_hits"] == 1
    # Assert expected behavior to validate correctness.
    assert LOOKUP_ATTEMPTS["billing"] == 1
    # Assert expected behavior to validate correctness.
    assert len(_tool_result_blocks(result["messages"])) == 2

    # Existing inline note.
    # 2) Retry behavior for transient errors
    # Call this function to perform the next operation.
    reset_state()
    # Assign computed data to a named variable for later use.

```

```

model = TriageModel("retry")
# Assign computed data to a named variable for later use.
result = run_agent("trriage payments", model, TOOL_REGISTRY)
# Assert expected behavior to validate correctness.
assert LOOKUP_ATTEMPTS["payments"] == 2
# Assert expected behavior to validate correctness.
assert result["final"]["action"] == "restart_payments_workers"

# Existing inline note.
# 3) Structured final output required
# Assert expected behavior to validate correctness.
assert {"summary", "action", "confidence"}.issubset(result["final"].keys)

# Existing inline note.
# 4) Claude sequencing: assistant tool_use immediately followed by user
# Call this function to perform the next operation.
assistant_idx = next(i for i, m in enumerate(result["messages"]) if m["role"] == "assistant")
# Assert expected behavior to validate correctness.
assert result["messages"][assistant_idx + 1]["role"] == "user"
# Assert expected behavior to validate correctness.
assert all(
    # Execute this line as part of the solution flow.
    block.get("type") == "tool_result" for block in result["messages"][assistant_idx + 1:]
)

# Existing inline note.
# 5) Max step protection
# Assign computed data to a named variable for later use.
model = TriageModel("loop")
# Start guarded block to handle potential runtime errors.
try:
    # Assign computed data to a named variable for later use.
    run_agent("loop", model, TOOL_REGISTRY, max_steps=3)
    # Raise explicit error to fail fast on invalid state.
    raise AssertionError("Expected max_steps_exceeded")
# Handle expected failure path and keep behavior predictable.
except RuntimeError as exc:
    # Assert expected behavior to validate correctness.
    assert "max_steps_exceeded" in str(exc)

# Call this function to perform the next operation.
print("03_mock tests passed")

# Call this function to perform the next operation.
run_exam03_tests()

```